

6p

```
let userName: string = "kim";  
let age: number = 20;  
let isLogin: boolean = true;
```

```
let selected: string | null = null; // 유니온 타입 문자열이나 null을 저장가능  
let result: // undefined
```

10p

```
const user = "홍길동";  
const age = 20;
```

```
console.log(`이름은 ${user}이고 나이는 ${age}살 입니다.`);
```



```
npx.cmd tsx "c:\Urs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\w4\w4_10p.ts"  
이름은 홍길동이고 나이는 20살 입니다.
```

12p

```
const numString = "100";  
const num = parseInt(numString);
```

```
console.log(typeof(numString));  
console.log(numString);  
console.log(typeof(num));  
console.log(num);
```

```
const a = "10";  
const b = "20";
```

```
const result = Number(a) + Number(b);  
console.log("a + b = " + result);
```

```
const value = Number("hello");
```

```
console.log(value);  
console.log(typeof value);
```

```
PS C:\Users\USER\Desktop\sophomore_2ndTerm\Frontend_Programing> npx.cmd tsx "c:\Users\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\w4\w4_12p.ts"
number
100
a + b = 30
NaN
number
```

13p

```
console.log(100 / 0);
console.log(-1 / 0);
console.log(Infinity - Infinity);
```

```
npx.cmd tsx "c:\Users\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\w4\w4_13p.ts"
Infinity
-Infinity
NaN
```

23p javascript

```
let name = "Kim"; // string
let age = 20; // number
let student = true; // boolean
let value; // undefined
let data = null; // null
let id = Symbol("id"); // symbol
let big = 12345678901234567890n; // bigint
let user = {
  name: "Kim",
  age: 20
}; // Object
```

23p typescript

```
let arr = [1, 2, 3] // 배열
console.log(typeof arr);

function hello() { // 함수
  console.log("Hello");
}

console.log(typeof hello); // 객체 성격이지만 출력 결과는 function

let user = { // Object
  name: "Kim",
  age: 20
};
console.log(typeof user);
```

```
npx.cmd tsx "c:\U
rs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\W4\w4_23p.ts"
object
function
object
```

24p

```
let value:any = 10;
```

```
console.log(typeof value);
```

```
value = "hello";
```

```
console.log(typeof value);
```

```
value = true;
```

```
console.log(typeof value);
```

```
value = [1, 2, 3];
```

```
console.log(typeof value);
```

```
npx.cmd tsx "c:\U
rs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\W4\w4_24p.ts"
number
string
boolean
object
```

25p

```
let value : unknown;
```

```
value = 10;
```

```
value = "hello";
```

```
if (typeof value == "string") {
    console.log(value.toUpperCase());
}
```

```
npx.cmd tsx "c:\U
rs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\W4\w4_25p.ts"
HELLO
```

26p

```
function hello(): void {
    console.log("Hello");
}
```

```
function error(message: string): never {
    throw new Error(message);
}
```

27p

```
// 일반 배열
let userArray : (string | number)[] = ["Kim", 20];
// Tuple - 위치별 타입이 결정
// index 0 : string
// index 1 : number
let userTuple: [string, number] = ["Kim", 20];
// let wrongUser: [string, number] = [20, "Kim"]; // 오류
```

```
28p
// 열거형
enum Direction {
    Up,
    Down,
    Left,
    Right
}
let direction: Direction = Direction.Right;
console.log(direction); // 해당값의 인덱스 번호를 출력

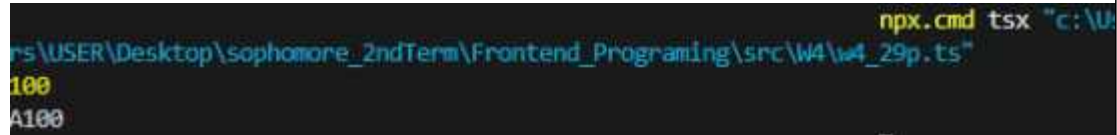
enum Role1 {
    Admin = "ADMIN",
    User = "USER",
    Guest = "GUEST"
}

let userRole1: Role1 = Role1.Admin;
console.log(userRole1);
let userRole2: Role2 = Role1.Guest;
console.log(userRole2);
type Role2 = "ADMIN" | "USER" | "GUEST";
```

```
npx.cmd tsx "c:\U
rs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\W4\w4_28p.ts"
3
ADMIN
GUEST
```

```
29p
let id: string | number: // string 또는 number
id = 100;
id = "A100";
function printId(id: string | number) {
    console.log(id);
}
id = 100;
```

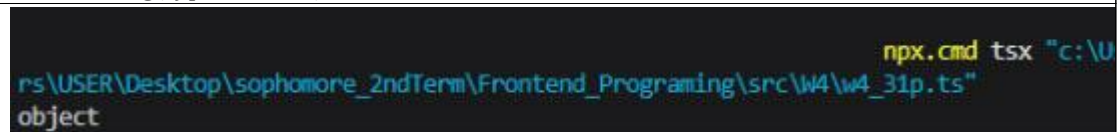
```
printId(id);  
id = "A100";  
printId(id);
```



```
npx.cmd tsx "c:\U...  
rs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\W4\w4_29p.ts"  
100  
A100
```

```
30p  
let direction: "left" | "right";  
direction = "left";  
direction = "right";  
// direction = "up"; // 오류 둘 중 하나만 사용 가능함  
type Status = "loading" | "success" | "error" ; // React와 Web API의 사용 예
```

```
31p  
type Person = {  
    name: string;  
};  
type Employee = {  
    employeeId: number;  
};  
type Worker = Person & Employee; // Person + Employee -> Worker  
  
const user: Worker = {  
    name: "Kim",  
    employeeId: 1001  
};  
  
console.log(typeof user);
```



```
npx.cmd tsx "c:\U...  
rs\USER\Desktop\sophomore_2ndTerm\Frontend_Programing\src\W4\w4_31p.ts"  
object
```

```
32p  
interface User {  
    name : string;  
    age : number;  
}  
  
const user: User = {  
    name : "Kim",  
    age : 20  
};
```

```

interface User1 {
    name : string;
    age : number;
    email ? : string; // 선택적 속성
}

const user1: User1 = {
    name : "kim",
    age : 20
};

const user2 : User1 = {
    name : "park",
    age : 20,
    email : "parksuejin@naver.com"
};

interface User2 {
    readonly id : number;
    name : string;
}

const user3 : User2 = {
    id: 1,
    name : "Lee"
};

user3.name = "Lee";
// user.id = 2; // readonly라 불가능

```

33p

```

type UserName = string; // 특정 타입에 새로운 이름을 붙이는 방법
let name: UserName = "Kim"; // UserName 은 string 타입임

type User = {
    name: string;
    age: number;
};

const user: User = {
    name: "Kim",
    age: 20
};

```